Client-Server Architecture on the Web

Course notes - Department of Computer Science

1. The request-response cycle

A browser resolves a hostname, opens a connection, sends an HTTP request and receives a response. The method conveys intent - GET to read, POST to create, PUT to replace, PATCH to modify, DELETE to remove - and the status code conveys the outcome.

GET and HEAD must be safe, meaning free of observable side effects. GET, PUT and DELETE must be idempotent: repeating them leaves the same state, which is what makes retries safe.

2. REST API design

A REST resource is identified by a URL and manipulated through the uniform interface. Paths name nouns and the method supplies the verb, so /students/42/submissions is preferred over /getStudentSubmissions.

Return 200 for a successful read, 201 with a Location header when creating, 400 for malformed input, 401 when unauthenticated, 403 when authenticated but not permitted, and 404 when the resource does not exist. Using 200 with an error body defeats every intermediary that inspects status codes.

3. Authentication and sessions

HTTP is stateless, so identity must be carried on each request. Server-side sessions store state keyed by a cookie; token schemes such as JWT carry signed claims in the request itself, trading revocability for statelessness.

Store tokens carefully. Anything readable by JavaScript is exposed to cross-site scripting; cookies marked HttpOnly and SameSite resist that but require attention to cross-site request forgery.

4. The browser security model

The same-origin policy isolates documents by scheme, host and port. Cross-origin resource sharing relaxes it explicitly: the server names permitted origins, and the browser enforces the decision.

Cross-site scripting is prevented by escaping untrusted data at the point of output and by a content security policy. SQL injection is prevented by parameterised queries. Both share a root cause, which is treating data as code.

5. Client-side rendering

Single-page applications render in the browser and fetch data asynchronously, which makes navigation feel immediate but shifts cost onto the client and complicates initial load and indexing.

Server-side rendering returns markup ready to display and hydrates it afterwards. The choice is a trade between time-to-first-byte, time-to-interactive and operational complexity, and it should follow the application's actual audience.